

pynxtools: A Python framework for generating and validating NeXus files in experimental data workflows

Sherjeel Shabih ^{1*}, Lukas Pielsticker ^{1,2*}, Florian Dobener ^{1,3}, Andrea Albino ¹, Theodore Chang ¹, Carola Emminger ^{1,4}, Lev Ginzburg ¹, Ron Hildebrandt ^{1,4}, Markus Kühbach ¹, Rubel Mozumder ¹, Tommaso Pincelli ⁵, Martin Aeschlimann ³, Marius Grundmann ⁴, Walid Hetaba ², Carlos-Andres Palma ^{1,6}, Laurenz Rettig ⁵, Markus Scheidgen ¹, Sandor Brockhauser ¹, José Antonio Márquez Prieto ¹, Claudia Draxl ¹, Christoph T. Koch ¹, and Heiko B. Weber ⁷

¹ Physics Department and CSMB, Humboldt-Universität zu Berlin, Zum Großen Windkanal 2, D-12489 Berlin, Germany  ² Department Heterogeneous Reactions, Max Planck Institute for Chemical Energy Conversion, Stiftstraße 34-36, D-45470 Mülheim an der Ruhr, Germany  ³ Department of Physics, RPTU Kaiserslautern-Landau, Erwin-Schrödinger-Str. 46, D-67663 Kaiserslautern, Germany  ⁴ Felix Bloch Institute for Solid State Physics, Leipzig University, Linnestr. 5, D-04103 Leipzig, Germany  ⁵ Department of Physical Chemistry, Fritz Haber Institute of the Max Planck Society, Faradayweg 4-6, D-14195 Berlin, DE  ⁶ Institute of Physics, Chinese Academy of Sciences, No.8, 3rd South Street, Zhongguancun, Haidian District, Beijing, China  ⁷ Lehrstuhl für Angewandte Physik, Friedrich-Alexander-Universität Erlangen-Nürnberg, Staudtstr. 7, D-91058 Erlangen, Germany  * These authors contributed equally.

DOI: [10.21105/joss.10691](https://doi.org/10.21105/joss.10691)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Blake Rayfield](#) 

Reviewers:

- [@Grant-Xue](#)
- [@marjanalbooyeh](#)

Submitted: 14 October 2025

Published: 07 October 2026

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Scientific data across physics, materials science, and materials engineering often lacks adherence to FAIR principles ([Barker et al., 2022](#); [Jacobsen et al., 2020](#); [M. D. Wilkinson et al., 2016](#); [S. R. Wilkinson et al., 2025](#)) due to incompatible instrument-specific formats and diverse standardization practices. pynxtools is a Python software development framework with a command line interface (CLI) that standardizes data conversion for scientific experiments to the NeXus format ([Klosowski et al., 1997](#); [Könnecke, 2006](#); [Könnecke et al., 2015](#)) across diverse scientific domains including atom probe tomography, electron microscopy, optical spectroscopy, photoemission spectroscopy, scanning probe microscopy, and X-ray diffraction. Through its modular plugin architecture, pynxtools converts data and metadata from instruments and electronic lab notebooks to unified NeXus application definitions, which are standardized schemas that specify how data and metadata for a particular type of experiment are organized within NeXus, while performing automated validation to ensure compliance. The framework is deployed in production as part of the NOMAD ([Draxl & Scheffler, 2019](#); [Scheidgen et al., 2023](#)) research data management platform and is used across multiple NOMAD Oasis installations in experimental laboratories, by external Python packages, and in dataset publications on Zenodo.

Statement of Need

Achieving FAIR (Findable, Accessible, Interoperable, and Reproducible) data principles in experimental physics and materials science requires consistent implementation of standardized data formats. NeXus provides comprehensive specifications for structured storage of scientific data through technique-specific application definitions. However, leveraging its full breadth and detailed NXDL documentation can require careful implementation. pynxtools addresses

this by transforming NeXus from a complex specification into a practical solution through guided workflows, automated validation with detailed error reporting for missing required data points, and clear implementation pathways via configuration files and extensible plugins. This approach enables researchers to achieve true data interoperability without deep technical expertise in the underlying standards.

State of the Field

Existing tools in the NeXus ecosystem typically address isolated aspects of adoption. For example, `nxconvert` supports backend conversion between NeXus representations (e.g., HDF5 to XML), `nxbrowse` enables inspection of file contents, and `cnxvalidate` (Könnecke et al., 2024) together with `punx` (Jemian et al., 2025) provide validation of NeXus files. In addition, some domain-specific software packages implement technique-specific routines for writing NeXus files.

`pynxtools` fills this gap by dynamically integrating any version of NXDL application definitions directly from their source repositories and validating conversions against the complete inheritance structure of the chosen definition. Rather than just offering an additional reader/writer, it operationalizes NeXus as an executable specification with automated, exhaustive validation and structured error reporting. The validation layer is paired with a plugin ecosystem that enables format-to-NeXus conversion for diverse instruments. Contributing comparable functionality to existing tools would require substantial architectural modifications, whereas `pynxtools` was, from the outset, designed as a dedicated framework that provides a cohesive, extensible solution to transform NeXus adoption from a manual, documentation-heavy task into a guided, programmatic workflow.

Software Design

The core design decision in `pynxtools` is to treat NeXus application definitions not as static documentation but as machine-actionable schemas that drive the entire conversion workflow. We chose a plugin-based architecture to decouple instrument-specific parsing logic from specification-aware validation. Each plugin declares the file formats it supports and the NXDL application definitions it can target, enabling modular growth without central code modification. This design supports an evolving ecosystem where new techniques and file formats can be added independently.

A key architectural choice is the internal `Template` object: a subclass of Python's native dictionary that serves as a form for reader plugins to fill. By preserving the standard Python dict API, developers interact with a familiar data structure while benefiting from deep validation features. The `Template` embeds structural awareness of required, recommended, and optional NeXus fields, pre-populating and organizing them according to the selected application definition. Because the structure maps naturally to JSON, it can be serialized, inspected, and edited outside the runtime environment, supporting tools such as JSON/YAML mappers and electronic lab notebook integration.

`dataconverter`, the core module of `pynxtools` that provides the transformation workflow, validates reader output against the selected NeXus application definition, checking for instances of required concepts, complex dependencies (like inheritance and nested group rules), and data integrity (type, shape, constraints). All reader plugins are tested using the `pynxtools.testing` suite, which runs automatically via GitHub CI to ensure compatibility with the `dataconverter`, the NeXus specification, and integration across plugins.

This modularity comes with some additional complexity compared with implementing instrument-specific conversion logic directly in a monolithic workflow, particularly in managing plugin dependencies and maintaining consistency across extensions. Likewise, schema-driven

validation and the construction of the Template require additional processing compared with simpler, format-specific conversion approaches. We accepted these costs because they provide a common validation framework, early detection of structural and data errors, and greater interoperability and maintainability as the number of supported techniques and file formats grows.

Implementation

The dataconverter combines instrument output files and data from electronic lab notebooks into NeXus-compliant HDF5 files. It provides a command-line interface (CLI) supporting both built-in readers for general-purpose functionality and technique-specific reader plugins distributed as separate Python packages. For developers, the dataconverter provides an abstract reader class for building plugins that process experiment-specific formats and populate the NeXus specification through the Template interface.

The dataconverter includes a tool called `eln_mapper` that creates either a fillable YAML file or a NOMAD ELN schema based on a selected NeXus application definition.

The `read_nexus` module enables semantic access to NeXus files by linking data items to NeXus concepts, allowing applications to locate relevant data without hardcoding file paths. It supports concept-based queries that return all data items associated with a specific NeXus vocabulary term. Each data item is annotated by traversing its group path and resolving its corresponding NeXus concept, including inherited definitions. Items not part of the NeXus schema are explicitly marked as such, aiding in validation and debugging. The tool also identifies and summarizes the file's default plottable data based on the NXdata definition.

Research Impact

`pynxtools` is integrated into production systems, demonstrating real-world impact across multiple deployment contexts. It is designed as a plugin for NOMAD to support NeXus and the features that come with our framework. Within the NOMAD platform, it underpins NeXus-based ingestion, validation, and search of experimental datasets, enabling automatic unit handling, metadata linking, and advanced search over NeXus-defined quantities. The framework is also distributed with NOMAD Oasis installations operated in experimental laboratories, demonstrating its suitability for decentralized, on-premise data management.

The NOMAD integration consists of several key components: the NOMAD's data schema (MetaInfo (Ghiringhelli et al., 2017)) is extended by integrating NeXus definitions as a Schema Package, introducing NeXus-specific quantities and enabling interoperability with other standardized data representations. The NexusParser module within `pynxtools` extracts structured data from NeXus HDF5 files to populate NOMAD with MetaInfo object instances. Parsed data is post-processed using NOMAD's Normalization pipeline, including automatic handling of units, linking references (sample and instrument identifiers), and populating derived quantities needed for advanced search and visualization. An integrated Search Application powered by Elasticsearch (Elasticsearch B.V., 2025) provides a dashboard for filtering uploaded data based on experiment type, upload timestamp, and domain-specific quantities. The entire workflow is exemplified in a representative Example Upload shipped with the package.

Beyond NOMAD, `pynxtools` supports external Python packages that rely on its validation and conversion capabilities and has been used in preparing datasets for publication on repositories such as Zenodo (Beaulieu, Schöler, et al., 2021; Beaulieu, Dong, et al., 2021; Dendzik & others, 2022; Julian Maklar et al., 2020, 2022; J. Maklar et al., 2023; Pincelli et al., 2022; Xian et al., 2020). Its continuous integration testing across plugins ensures robustness against updates in both NXDL definitions and dependent packages, signaling community readiness and maintainability. By lowering the technical barrier to rigorous NeXus compliance, `pynxtools`

enables reproducible, interoperable experimental data workflows that are already operational across multiple scientific domains.

Funding

The work is funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) - project 460197019 (FAIRmat).

Acknowledgements

We acknowledge the following software packages our package depends on: h5py (Collette & al., 2008), numpy (Harris et al., 2020), click (The Pallets development team, 2014), CFF (Druskat et al., 2021), xarray (S. Hoyer & Hamman, 2017), (Stephan Hoyer et al., 2025), pandas (The pandas development team, 2020), scipy (McKinney, 2010), lxml (Behnel et al., 2005), mergedeep (Clarke, 2019), Atomic Simulation Environment (Larsen et al., 2017), pint (The Pint development team, 2012).

AI Usage Disclosure

Most of pynxtools was developed before AI assistance became part of the project's development workflow. More recent development has made use of AI-assisted software engineering tools, including Claude Code and GitHub Copilot, for tasks such as exploratory code generation, refactoring, and documentation updates. Development decisions, software design, and acceptance of contributions have so far remained under human control. All AI-generated content has been reviewed, modified where necessary, and validated by the authors prior to integration into the codebase.

References

- Barker, M., Chue Hong, N. P., Katz, D. S., Lamprecht, A.-L., Martinez-Ortiz, C., Psomopoulos, F., Harrow, J., Castro, L. J., Gruenpeter, M., & Martinez, P. A. et. al. (2022). Introducing the FAIR principles for research software. *Scientific Data*, 9(1), 622. <https://doi.org/10.1038/s41597-022-01710-x>
- Beaulieu, S., Dong, S., Tancogne-Dejean, N., Dendzik, M., Pincelli, T., Maklar, J., Xian, R. P., Sentef, M. A., Wolf, M., Rubio, A., Rettig, L., & Ernstorfer, R. (2021). *Time- and angle-resolved photoemission spectroscopy (trARPES) data of td-MoTe2 across an ultrafast dynamical lifshitz transition* [Data set]. Zenodo. <https://doi.org/10.5281/zenodo.4632480>
- Beaulieu, S., Schöler, M., Schusser, J., Dong, S., Pincelli, T., Maklar, J., Neef, A., Reinert, F., Wolf, M., Rettig, L., Minár, J., & Ernstorfer, R. (2021). *Multidimensional XUV photoemission spectroscopy of 1T-TiTe2 for different crystal orientations* [Data set]. Zenodo. <https://doi.org/10.5281/zenodo.5541489>
- Behnel, S., Faassen, M., & Bicking, I. (2005). *lxml: XML and HTML with python*. <https://lxml.de>
- Clarke, T. (2019). *Mergedeep: A deep merge function for python*. <https://github.com/clarketm/mergedeep>
- Collette, A., & al., et. (2008). *h5py: HDF5 for python*. <https://doi.org/10.5281/zenodo.6575970>
- Dendzik, M., & others. (2022). *Synchrotron bulk photoemission data from au(111) and DFT calculations* [Data set]. Zenodo. <https://doi.org/10.5281/zenodo.7305241>

- Draxl, C., & Scheffler, M. (2019). The NOMAD laboratory: From data sharing to artificial intelligence. *Journal of Physics: Materials*, 2(3), 036001. <https://doi.org/10.1088/2515-7639/ab13bb>
- Druskat, S., Spaaks, J. H., Chue Hong, N., Haines, R., Baker, J., Bliven, S., Willighagen, E., Pérez-Suárez, D., & Konovalov, O. (2021). *Citation file format*. <https://doi.org/10.5281/zenodo.1003149>
- Elasticsearch B.V. (2025). Elasticsearch-py: Official python client for elasticsearch. In *GitHub repository*. GitHub. <https://github.com/elastic/elasticsearch-py>
- Ghiringhelli, L. M., Carbogno, C., Levchenko, S., Mohamed, F., Huhs, G., Lüders, M., Oliveira, M., & Scheffler, M. (2017). Towards efficient data exchange and sharing for big-data driven materials science: Metadata and data formats. *Npj Computational Materials*, 3(1), 46. <https://doi.org/10.1038/s41524-017-0048-5>
- Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hoyer, S., & Hamman, J. (2017). Xarray: N-D labeled arrays and datasets in Python. *J. Open Res. Software*. <https://doi.org/10.5334/jors.148>
- Hoyer, Stephan, Roos, M., Joseph, H., Magin, J., Cherian, D., Fitzgerald, C., Hauser, M., Fujii, K., Maussion, F., Imperiale, G., Clark, S., Kleeman, A., Nicholas, T., Kluyver, T., Westling, J., Munroe, J., Amici, A., Barghini, A., Banihirwe, A., ... Littlejohns, O. (2025). Xarray (Version v2025.03.1). Zenodo. <https://doi.org/10.5281/zenodo.15110615>
- Jacobsen, A., Miranda Azevedo, R. de, Juty, N., Batista, D., Coles, S., Cornet, R., Courtot, M., Crosas, M., Dumontier, M., & Evelo, C. T. et. al. (2020). FAIR principles: Interpretations and implementation considerations. *Data Intelligence*, 2(1-2), 10–29. https://doi.org/10.1162/dint_r_00024
- Jemian, P. R., Ching, D., De Nolf, W., & Stöckli, P. (2025). *Prjemian/punx*. <https://doi.org/10.5281/zenodo.1471686>
- Klosowski, P., Koennecke, M., Tischler, J. Z., & Osborn, R. (1997). NeXus: A common format for the exchange of neutron and synchrotron data. *Physica B: Condensed Matter*, 241-243, 151–153. [https://doi.org/10.1016/S0921-4526\(97\)00865-X](https://doi.org/10.1016/S0921-4526(97)00865-X)
- Könnecke, M. (2006). The state of the NeXus data format. *Physica B: Condensed Matter*, 385-386, 1343–1345. <https://doi.org/10.1016/j.physb.2006.06.106>
- Könnecke, M., Akeroyd, F. A., Bernstein, H. J., Brewster, A. S., Campbell, S. I., Clausen, B., Cottrell, S., Hoffmann, J. U., Jemian, P. R., & Männicke, D. et. al. (2015). The NeXus data format. *Applied Crystallography*, 48(1), 301–305. <https://doi.org/10.1107/S1600576714027575>
- Könnecke, M., Bernstein, H. J., Richter, T., Caswell, T. A., Jemian, P. R., & Levinsen, Y. (2024). *Nexusformat/cnxvalidate*. <https://github.com/nexusformat/cnxvalidate>
- Larsen, A. H., Mortensen, J. J., Blomqvist, J., Castelli, I. E., Christensen, R., Duřak, M., Friis, J., Groves, M. N., Hammer, B., Hargus, C., Hermes, E. D., Jennings, P. C., Jensen, P. B., Kermode, J., Kitchin, J. R., Kolsbjerg, E. L., Kubal, J., Kaasbjerg, K., Lysgaard, S., ... Jacobsen, K. W. (2017). The atomic simulation environment—a python library for working with atoms. *Journal of Physics: Condensed Matter*, 29(27), 273002. <https://doi.org/10.1088/1361-648X/aa680e>
- Maklar, Julian, Dong, S., Beaulieu, S., Pincelli, T., Dendzik, M., Windsor, Y. W., Xian, R. P., Wolf, M., Ernstorfer, R., & Rettig, L. (2020). *Time-resolved ARPES data of bulk WSe2*

- for a quantitative comparison of time-of-flight momentum microscopes and hemispherical analyzers [Data set]. Zenodo. <https://doi.org/10.5281/zenodo.4067967>
- Maklar, Julian, Dong, S., Beaulieu, S., Pincelli, T., Dendzik, M., Windsor, Y. W., Xian, R. P., Wolf, M., Ernstorfer, R., & Rettig, L. (2022). *Time-resolved ARPES RAW data of bulk WSe₂ for a quantitative comparison of time-of-flight momentum microscopes and hemispherical analyzers: RAW MM data* [Data set]. Zenodo. <https://doi.org/10.5281/zenodo.6369727>
- Maklar, J., Sarkar, J., Dong, S., Gerasimenko, Y. A., Pincelli, T., Beaulieu, S., Kirchmann, P. S., Sobota, J. A., Yang, S.-L., Leuenberger, D., Moore, R. G., Shen, Z.-X., Wolf, M., Mihailovic, D., Ernstorfer, R., & Rettig, L. (2023). *Time- and angle-resolved photoemission data of 1T-TaS₂* [Data set]. Zenodo, Dataset; Zenodo. <https://doi.org/10.5281/zenodo.8238530>
- McKinney, Wes. (2010). Data Structures for Statistical Computing in Python. In Stéfan van der Walt & Jarrod Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56–61). <https://doi.org/10.25080/Majora-92bf1922-00a>
- Pincelli, T., Vasileiadis, T., Dong, S., Beaulieu, S., Dendzik, M., Zahn, D., Lee, S.-E., Seiler, H., Qi, Y., Xian, R. P., Maklar, J., Coy, E., Mueller, N. S., Okamura, Y., Reich, S., Wolf, M., Rettig, L., & Ernstorfer, R. (2022). *Time and momentum resolved characterization of hybrid plasmonic heterostructure au/WSe₂* [Data set]. Zenodo. <https://doi.org/10.5281/zenodo.7441231>
- Scheidgen, M., Himanen, L., Ladines, A. N., Sikter, D., Nakhaee, M., Fekete, Á., Chang, T., Golparvar, A., Márquez, J. A., Brockhauser, S., Brückner, S., Ghiringhelli, L. M., Dietrich, F., Lehmborg, D., Denell, T., Albino, A., Näsström, H., Shabih, S., Dobener, F., ... Draxl, C. (2023). NOMAD: A distributed web-based platform for managing materials science research data. *Journal of Open Source Software*, 8(90), 5388. <https://doi.org/10.21105/joss.05388>
- The Pallets development team. (2014). *Click: Command line interface creation kit*. <https://github.com/pallets/click>
- The pandas development team. (2020). *Pandas-dev/pandas: pandas*. Zenodo. <https://doi.org/10.5281/zenodo.3509134>
- The Pint development team. (2012). *Pint: Operate and manipulate physical quantities in Python*. <https://github.com/hgrecco/pint>. <https://github.com/hgrecco/pint>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., Silva Santos, L. B. da, & Bourne, P. E. et. al. (2016). The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3(1), 1–9. <https://doi.org/10.1038/sdata.2016.18>
- Wilkinson, S. R., Aloqalaa, M., Belhajjame, K., Crusoe, M. R., Paula Kinoshita, B. de, Gadelha, L., Garijo, D., Gustafsson, O. J. R., Juty, N., Kanwal, S., Khan, F. Z., Köster, J., Peters-von Gehlen, K., Pouchard, L., Rannow, R. K., Soiland-Reyes, S., Soranzo, N., Sufi, S., Sun, Z., ... Goble, C. (2025). Applying the FAIR principles to computational workflows. *Sci. Data*, 12(1), 328. <https://doi.org/10.1038/s41597-025-04451-9>
- Xian, R. P., Acremann, Y., Agustsson, S. Y., Dendzik, M., Bühlmann, K., Curcio, D., Kutnyakhov, D., Pressacco, F., Heber, M., Dong, S., Pincelli, T., Demsar, J., Wurth, W., Hofmann, P., Wolf, M., Scheidgen, M., Rettig, L., & Ernstorfer, R. (2020). *Datasets for the computational workflow of multidimensional photoemission spectroscopy* [Data set]. Zenodo. <https://doi.org/10.5281/zenodo.3987303>